

Tokenización: cómo "entienden" texto los modelos

Módulo 1 · Fundamentos Ontológicos y Razonamiento de Máquinas

BPE, vocabularios, coste por token, diferencias entre idiomas y por que los LLMs no pueden contar letras dentro de un token.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Tokenización: cómo "entienden" texto los modelos

Uno de los malentendidos más frecuentes sobre los LLMs es creer que leen texto como un humano: palabra por palabra, letra por letra, con comprensión del significado de cada unidad. No lo hacen. Los modelos procesan secuencias de tokens —fragmentos numéricos de texto— cuya segmentación responde a la frecuencia estadística en el corpus de entrenamiento, no a la estructura lingüística humana. Esta diferencia, aparentemente técnica, tiene consecuencias prácticas directas en el coste de las APIs, en la capacidad del modelo para contar letras, y en por qué los modelos se comportan de forma diferente con diferentes idiomas o tipos de texto.

La librería de tokenizers de Hugging Face, usada en prácticamente todos los modelos modernos, implementa los algoritmos BPE, WordPiece y Unigram con una velocidad de menos de 20 segundos para tokenizar un gigabyte de texto en un CPU de servidor. Esta velocidad no sería posible con algoritmos basados en palabras o caracteres para vocabularios del tamaño de los LLMs actuales.

La tokenización: de texto a IDs numéricos

Un tokenizador toma texto crudo y lo convierte en una secuencia de enteros que representan tokens en el vocabulario del modelo. Cada token tiene un ID único (un entero entre 0 y el tamaño del vocabulario). Estos IDs son los que el modelo realmente procesa: los convierte en vectores de embedding, los pasa por el Transformer, y produce una distribución de probabilidad sobre todos los IDs del vocabulario para el siguiente token.

El proceso inverso (del ID al texto) es la decodificación. Los tokenizadores modernos como el de GPT-4 (cl100k_base) o el de GPT-4o (o200k_base) tienen vocabularios de entre 50.000 y 200.000 tokens, representando fragmentos de texto comunes (palabras completas en inglés), subpalabras frecuentes, y bytes individuales como fallback.

Byte-Pair Encoding (BPE): el algoritmo dominante

BPE fue originalmente un algoritmo de compresión de datos (Philip Gage, 1994) adaptado para NLP por Sennrich, Haddow y Birch en 2016. El proceso de entrenamiento del tokenizador BPE comienza con el vocabulario de todos los bytes individuales (256 en byte-level BPE), y aplica iterativamente la regla: encuentra el par de tokens adyacentes más frecuente en el corpus, y fúndelos en un nuevo token. Repite hasta alcanzar el tamaño de vocabulario objetivo. GPT-2 aprendió 50.000 fusiones; GPT-4o aprendió ~200.000.

El resultado es un vocabulario donde las palabras comunes en inglés (like "the", "and", "is") son tokens únicos, las palabras comunes pero menos frecuentes se dividen en 2-3 tokens ("tokenization" → "token" + "ization"), y las palabras raras o no vistas se dividen en tokens más pequeños hasta llegar a bytes individuales si es necesario. Esto garantiza que cualquier texto puede ser tokenizado sin tokens desconocidos (zero unknown tokens).

Implicaciones prácticas de la tokenización

Coste: los tokens son la unidad de facturación

Las APIs de LLMs facturan por tokens de entrada y salida, no por palabras ni por caracteres. Una regla práctica aproximada: 1 token $\approx$ 0.75 palabras en inglés $\approx$ 4 caracteres. Pero esta aproximación falla con texto en otros idiomas (el español y el francés son generalmente 10-20% más caros en tokens que el inglés para el mismo contenido), código (más eficiente que el texto en prosa), números (altamente variable según el tokenizador), y caracteres especiales o matemáticos.

Problemas de tokenización: cuando el modelo "no ve" lo que esperarías

El famoso problema de "contar letras" en los LLMs tiene su raíz en la tokenización. Si "strawberry" se tokeniza como un único token (ID 12345), el modelo ve "12345", no S-t-r-a-w-b-e-r-r-y. Cuando le preguntas cuántas "r" hay, el modelo no puede contarlas porque no tiene acceso a la estructura interna del token; solo tiene acceso al ID entero. La solución: los modelos que reciben esta pregunta necesitan usar razonamiento extendido para descomponer el token, o el usuario puede preguntar "descompón la palabra en letras y luego cuenta".

El problema del idioma: los tokenizadores de los modelos actuales están fuertemente sesgados hacia el inglés porque fueron entrenados principalmente sobre texto en inglés. El mismo contenido en español, árabe, chino o hindi requiere entre 1.5x y 3x más tokens que en inglés, lo que se traduce directamente en un coste mayor y una ventana de contexto efectivamente más pequeña para esos idiomas.

SECCIÓN 4 · EJEMPLOS REALES

- Tokens y precios en APIs: GPT-4o cobra \$5 por millón de tokens de entrada. Un documento de 10 páginas en español tiene aproximadamente 25.000 tokens. Procesar 10.000 documentos al mes cuesta \$1.250 solo en tokens de entrada.
- Tokens en código: Python y JavaScript son relativamente eficientes en tokens (los identificadores de variables comunes son tokens únicos). SQL puede ser menos eficiente para sentencias largas. Código en lenguajes menos representados en el corpus de entrenamiento es sistemáticamente menos eficiente.
- Números: "12345" puede tokenizarse como un único token o como varios, dependiendo del tokenizador. Esto explica errores aritméticos en LLMs: el modelo no "ve" los dígitos individuales a menos que use razonamiento extendido o herramientas de cálculo.
- GPT-4 vocabulario: cl100k_base tiene ~100.000 tokens. GPT-4o usa o200k_base con ~200.000 tokens, lo que produce secuencias más cortas (menor coste) para el mismo texto al representar más fragmentos comunes como tokens únicos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: "Un token equivale a una palabra." No. En inglés, 1 token $\approx$ 0.75 palabras de media. En español y otros idiomas con morfología más rica, la proporción es peor. En código y en textos técnicos, la proporción varía enormemente.

Error 2: "La ventana de contexto es igual en todos los idiomas." No. Una ventana de 128K tokens en inglés equivale a ~100K tokens en español para el mismo volumen de información. Los sistemas multilingües deben planificar sus ventanas de contexto con esto en cuenta.

Error 3: "El tokenizador es transparente para el modelo." No lo es. El tokenizador afecta directamente qué información puede el modelo "ver" dentro de un token (nada, a menos que lo razone explícitamente) y cómo se segmentan los conceptos compuestos. Un tokenizador mal diseñado para un dominio (por ejemplo, médico o legal con mucha terminología especializada) puede degradar significativamente el rendimiento.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

La herramienta práctica más útil es tiktoken, la librería open source de OpenAI para tokenización: `pip install tiktoken`. Permite calcular exactamente cuántos tokens tiene cualquier texto con cualquier modelo de OpenAI. Para otros modelos, Hugging Face tokenizers proporciona los tokenizadores de Llama, Mistral, BERT y prácticamente todos los modelos del ecosistema.

Casos de uso prácticos del cálculo de tokens: estimar el coste de una API call antes de enviarla, verificar que un documento cabe en la ventana de contexto, comparar la eficiencia de diferentes idiomas o formatos para el mismo contenido, y optimizar prompts para reducir tokens sin perder información relevante.

Para sistemas que procesan grandes volúmenes de texto, la optimización de tokens puede tener impacto económico directo: eliminar espacios redundantes, usar formatos más compactos (JSON compacto vs JSON indentado), y evitar repetir instrucciones en cada llamada usando el prefix caching si el proveedor lo ofrece (Anthropic ofrece prompt caching con descuento del 90% en tokens de caché).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T8 (Transformer): los tokens son la unidad de entrada al Transformer. Sin entender la tokenización, no se puede entender el procesamiento del Transformer.
- M9 (Infraestructura): el coste de inferencia de un LLM está directamente determinado por el número de tokens procesados. La optimización de tokens es una forma directa de reducir costes de infraestructura.
- M6 (Ingeniería de Prompts): un prompt bien diseñado es también un prompt eficiente en tokens.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los LLMs no leen palabras ni caracteres: procesan secuencias de tokens, fragmentos de texto codificados como enteros. BPE (Byte-Pair Encoding) es el algoritmo dominante: comienza con bytes individuales y fusiona iterativamente los pares más frecuentes hasta alcanzar el vocabulario objetivo (50K-200K tokens). Los tokens son la unidad de facturación en APIs. El español es un 15-25% más caro que el inglés en tokens. Los números y el código tienen comportamientos especiales. Herramientas prácticas: tiktoken para OpenAI, Hugging Face tokenizers para el resto del ecosistema. Optimizar

tokens en sistemas de alto volumen tiene impacto económico directo y medible.